

LASR: IoT-Driven Autonomous Robot (The RockyBorg Project)

Ravi Verma

Internet of Things (IoT)

Westsächsische Hochschule Zwickau

ravi.verma.odf@whz.de

1 Abstract

LASR proposes an IoT-based architecture for an autonomous robot, which could track a line and respond effectively to obstacles. The system is organised into four layers, namely, Perception, Network, Processing, and Application. The Perception layer captures the visual data via the camera, while the Network layer is responsible for transferring this data between software modules and hardware components. The Processing layer performs the real-time image analysis, line and red obstacle detection, and calculates the steering based on PID control. The Application layer runs the motion of the robot by executing these decisions. The above design facilitates understanding and maintaining the system and upgrading it further. Overall, LASR illustrates how an autonomous robot's functions can be modularised to consistently achieve reliable performance.

Keywords: IoT-Based Robotics, Autonomous Navigation, Line Following, PID Control, Image Processing, Layered Architecture

2 Introduction

IoT has enabled the development of smarter autonomous systems, finding wide applications in logistic, agriculture, and intelligent transportation. A common requirement for these systems is reliable navigation; this is often realized by the implementation of vision-based line-following robots that can respond to obstacles in real time.

The RockyBorg project addresses this need by developing an autonomous line-following robot using the RockyBorg kit integrated with a Raspberry Pi 3B+. The platform enables DC motor control, servo steering, and camera-based navigation. Computer vision using OpenCV is employed in the detection of black lines, curves, junctions, and red obstacles, while the RockyBorg PCB and the camera module provide the core hardware infrastructure.

In order to organize the development and review process of the software system, the project employs the Lightweight Approach for Software Reviews. The

method is designed to be lightweight but structured, capable of conducting effective architectural reviews without overhead from traditional review processes. The system design follows a layered architecture: Perception, Network, Processing, and Application. This improves system clarity, efficiency in debugging, and makes it more scalable and maintainable.

The key quality attributes that LASR evaluates include reliability, maintainability, performance, robustness, and scalability. Potential risks are identified in a structured manner, while architecture gaps are quantified using spider charts. The evaluation approach supports iterative refinement in system design and helps to ensure that the eventual implementation meets both the functional and quality objectives for the autonomous robot.

3 Mission Statement

The LASR project's mission is to apply the Lightweight Approach of Software Review to the RockyBorg autonomous robot system. The project focuses on systematically reviewing, refining, and improving its software architecture. By reviewing its perception, processing, networking, and application layers, the LASR project intends for the RockyBorg's line-following, red-object detection, and PID-based steering functions to be reliable, maintainable, and well-structured. The goal is to demonstrate how lightweight software reviews can enhance the quality, clarity, and robustness of autonomous robotic systems.

4 System Architecture

System Architecture: To explain how the LASR system is structured throughout the four architectural layers: Perception, Network, Processing, and Application; and how data would flow through the components of the robot.

4.1 Perception Layer

Represented by the Camera View at the top.

The Pi camera captures the black line and environment in real time.

This is the raw visual data passed to the processing sys-

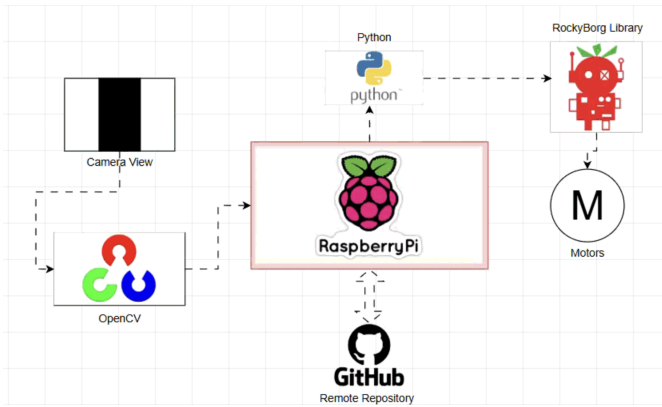


Figure 1: System Architecture

tem.

4.2 Network Layer

Shown as the dashed arrows connecting the Camera → OpenCV → Raspberry Pi → RockyBorg board.

This layer handles the communication between software modules and the hardware components.

This ensures that data flows smoothly from sensing to processing and finally to motor control.

4.3 Processing Layer

Represented by the OpenCV logo and the Python code block.

OpenCV runs on the Raspberry Pi, analyzing the camera feed, finding the black line, detecting obstacles, and computing PID steering values.

All the decision-making logic, including the line detection and control algorithms, occurs here.

4.4 Application Layer

Shown via the RockyBorg board and motor icon (M).

This layer performs the robot's actions, such as steering, going forward, slowing down, or stopping.

The Raspberry Pi issues commands to the RockyBorg board, which drives the motors.

4.5 GitHub Integration

The diagram also includes GitHub, which represents version control and usage of the remote repository for the source code of the project.

5 Quality Attributes

The LASR architecture emphasizes a number of key quality attributes that define the performance and reliability of the system:

5.1 Reliability

Reliability is a first-order attribute. The robot is supposed to repeatedly:

Detect the black line Identify and stop for red obstacles Curve and junction negotiation without failure

It achieves reliability through stable camera calibration, memory-based fallback logic, and refined PID tuning.

5.2 Maintainability

Maintenance is easy because the code is modularized into clear, independent modules. Since each module handles one type of function, updates are easily done, debugging is easily conducted, and new features are easily added without affecting the rest of the system.

5.3 Performance / Real-Time Response

The robot needs to process images in real-time, using efficient algorithms like ROI cropping, contour detection, and pixel-binning when tested. Real-time PID steering ensures smooth, responsive motion.

5.4 Scalability

The layered architecture allows new features such as additional sensors, different visual pipelines, or cloud-based IoT functionality to be integrated without re-designing the whole system.

5.5 Robustness

It is designed to cope with environmental changes such as lighting, shadows, glare, and partial visibility of the track. Techniques such as manual camera settings and memory-based prediction achieve very substantial gains in robustness.

6 Risk Management

Risk Assessment Table identifying various possible issues in the LASR system along with a risk rating based on three factors:

Likelihood of Occurrence

Impact

Priority

The purpose is to highlight which risks need immediate attention and which ones are less hazardous regarding system performance.

6.1 Lighting Changes Affecting the Camera

Chance: Medium

Impact: High

Priority: High

Strong shadows, bright reflections, and dim areas can easily interfere with the camera's ability to detect the

black line or red obstacles. This is a significant risk since the robot relies much on vision; thus, mitigation through better calibration or controlling the light is warranted.

6.2 PID Steering becoming unstable

Chance: High

Impact: High

Priority: Very High

PID instability may cause sudden jerks, overcorrections, or incorrect turning. Since steering directly impacts navigation accuracy, any instability significantly affects the results. This is marked as a high-priority risk and needs to be mitigated through careful PID tuning and testing.

6.3 Additional Lines visible in the ROI (confusing detection)

Chance: High

Impact: Medium

Priority: High

Additional unwanted lines within the camera's ROI such as shadows, tape edges, and track borders, can confuse the line detection logic. Although the robot may recover, the risk still demands attention to ensure consistent behavior.

6.4 Red-Object False Detection

Chance: Medium

Impact: Medium

Priority: Medium

Sometimes, the robot may detect non-red elements or shadows as a red obstacle. This causes extra stops, which are not necessary. While not as critical as PID and lighting risks, it still demands color filtering improvements.

6.5 Motor Spikes or Sudden Jerks

Chance: Low

Impact: Medium

Priority: Low

The risk for occasional electrical spikes or motor jerks exists but is slight. Although the impact is moderate, this issue is not as urgent due to the other risks discussed above.

7 Quality Analysis

Evaluates the LASR system using five key quality attributes: Reliability, Performance, Maintainability, Scalability, and Robustness, with a current score and target

score for each and a note about what needs improvement. It gives insight into how the system currently performs and where further refinement is needed.

7.1 Reliability – Score: 7/10 (Target: 10/10)

Reliability denotes the consistency of a robot executing line following and detecting obstacles. The current score of 7 indicates that the system works well but may still have some detection issues because of lighting or camera angle. Improvement Needed: Stronger camera setup and better line-detection logic.

7.2 Performance – Score: 7/10 (Target: 10/10)

Performance tells how efficiently the robot handles real-time processing and responds to track changes. A score of 7 indicates the system works, but under certain conditions, Rockyborg might slow down or delay its decisions. Improvement Needed: Better optimization for real-time performance.

7.3 Maintainability – Score: 9/10 (Target: 10/10)

Maintainability represents how easily the system can be updated, changed, and debugged. Improvement Needed: More documentation and even clearer structuring for the code to reach full maintainability.

7.4 Scalability – Score: 8/10 (Target: 10/10)

Scalability is how well this system can be augmented with new features, such as sensors and behaviors. An 8 means the design is scalable, but leaves room for enhancement. Improvement Needed: The design should be more flexible to accommodate future extensions.

7.5 Robustness – Score: 8/10 (Target: 10/10)

Robustness is the measure of how well the system handles variations in lighting, noise, or surface changes. An 8 means that the system is stable, but still sensitive to certain conditions. Improvement Needed: Improved filtering and enhanced preprocessing to reduce false detections.

Quality Analysis summarizes the strengths and weaknesses of the LASR system. The robot demonstrates a very good performance for maintainability, scalability, and robustness, but further improvements should be made toward ideal scores of 10/10, mainly for reliability and real-time performance, since these two topics are very critical for autonomous navigation.

8 Spider-Web Diagrams

The spider charts compare the ideal LASR target scores with the current performance of the system across five quality attributes: Reliability, Performance, Maintainability, Scalability, and Robustness.

LASR Benchmark – After Step 2 (Green Plot)

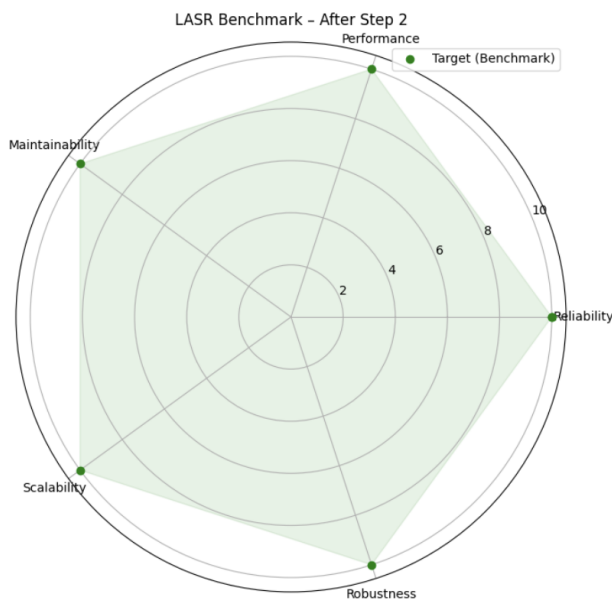


Figure 2: Target Values

Shows the target scores (10/10) for all quality attributes.

Represents the best performance after applying the LASR architecture.

Forms a perfect pentagon indicating maximum quality.

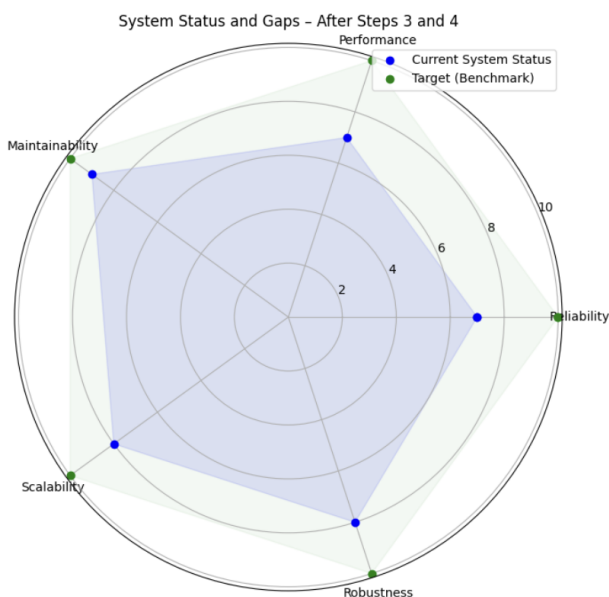


Figure 3: Current Values

System Status – After Steps 3 & 4 (Blue Plot)

Shows the actual performance after implementation and testing.

Strong areas:

Maintainability (9/10): Clean modular code and easy to update.

Robustness & Scalability (8/10): The system handles variations well and can be extended.

Areas needing improvement:

Reliability (7/10): Affected by lighting variation and detection stability.

Performance (7/10): Real-time processing can be further optimized.

9 Conclusion

The LASR methodology successfully provided a structured, lightweight, and effective way to review the RockyBorg robot's software architecture.

By separating the robot into Perception, Network, Processing, and Application layers, the project gained clearer modular design, easier debugging, and improved maintainability.

Quality analysis showed strong performance in Maintainability, Robustness, and Scalability, confirming the value of LASR in complex IoT-based robotic systems.

Identified weaknesses, especially Reliability and Real-Time Performance offer clear directions for future optimisation.

Overall, the project demonstrates how lightweight software reviews can enhance the clarity, reliability, and long-term evolution of autonomous robotic systems.

References

- [1] Embarc. (n.d.). *Architecture cheat sheet 1: Architecture overview* [PDF]. <https://blog.embarc.de/downloads/cheat-sheets/01/architecture-cheat-sheet-1-architecture-overview.pdf>
- [2] Fowler, M. (2003). *Who needs an architect?* [PDF]. IEEE Software. <https://martinfowler.com/ieeeSoftware/whoNeedsArchitect.pdf>
- [3] Embarc. (n.d.). *Leichtgewichtige Reviews* [Web page]. <https://www.embarc.de/architektur-spicker/12-leichtgewichtige-reviews/>
- [4] Verma, R. (n.d.). *IoT Project – RockyBorg* [PDF]. GitHub. https://github.com/RaviVerma05/RockyBorg/blob/main/IOT%20Project_Rockyborg.pdf